AGENTIC SDLC · НЕДЕЛЯ 2

Контекст и feedback loop

Минимальный контекст-пакет и цикл проверок, замкнутый на командах проекта

Релевантность | AGENTS.md | Состояние | Feedback






```
# контекст — это не «вся история»
context = system + rules + task + history + tools
```

План лекции

1

Релевантность контекста важнее объёма

Меньше, но точнее: что модель видит в одном вызове.

2

AGENTS.md и знания проекта

Правила, команды и границы — в файлах, а не в голове.

3

Состояние между сессиями

История временна, файлы постоянны: решения и журнал.

4

Feedback loop и порядок проверок

От дешёвых проверок к дорогим, цикл замыкается командой.

5

Как читать вывод команд

Код возврата, первая ошибка, полный лог.

6

Типичные ошибки и разобранный пример

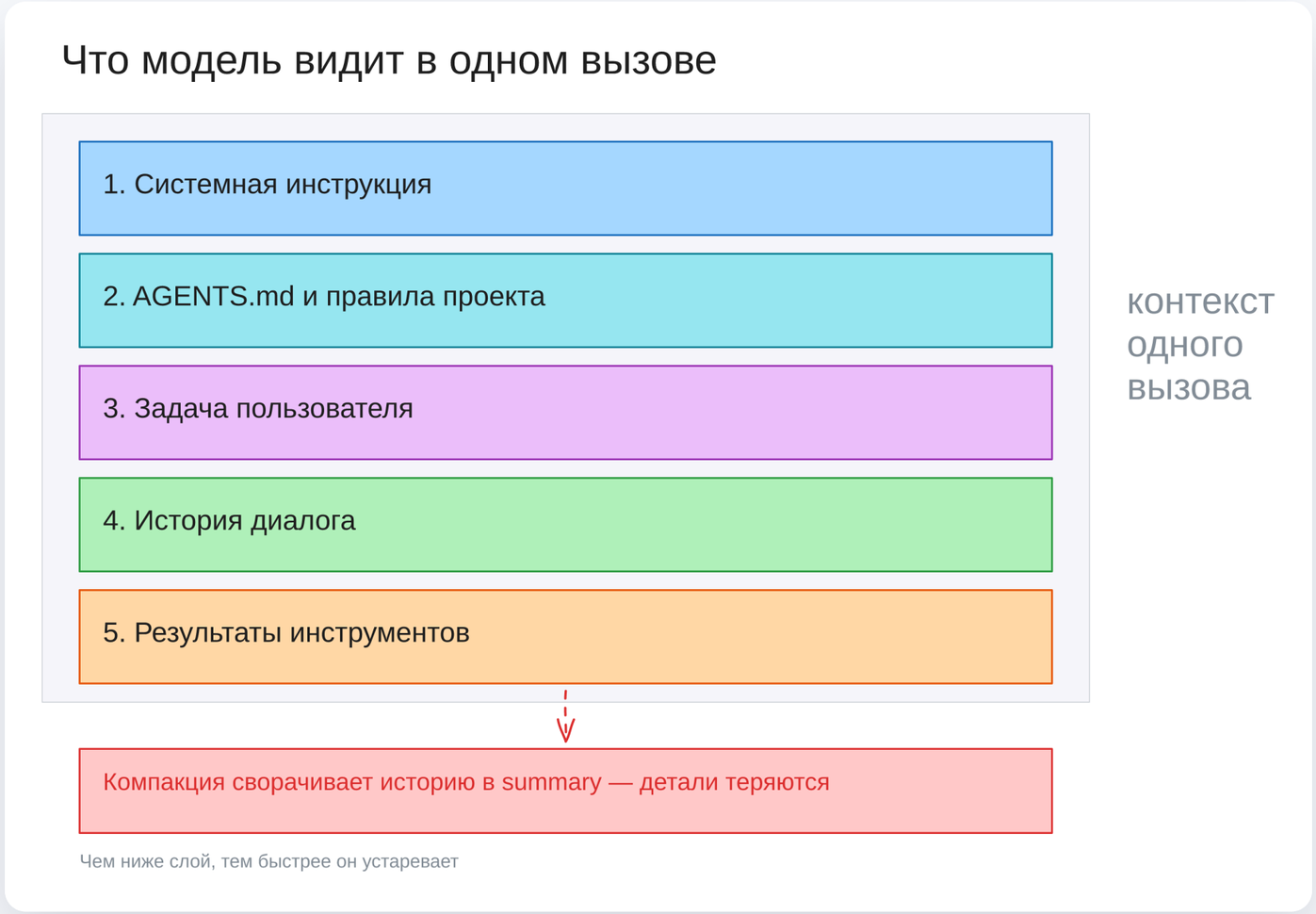
Пять ошибок, которые обесценивают проверку.

Цель: Минимальный контекст-пакет и цикл проверок, замкнутый на командах проекта

Контекст — минимум, без которого задача не решается

1. Постановка из недели 1
2. 3–7 файлов, которые участвуют в изменении
3. Спека или user story
4. Ограничения проекта
5. Команды проверки
6. Список неизвестностей

Пять слоёв контекста



Пять слоёв контекста

«Прочитать весь репозиторий» — плохая стратегия

1 **Окно контекста конечно, а полный проект дороже**

2 Внимание размывается: похожие файлы путают

3 Мёртвый код и старые спеки агент читает как руководство

Сначала карта и указатели, потом правки

1 **Карта: где backend, frontend, спеки, тесты**

2 Указатели: логика — файл, тест — файл, спека — путь

3 Неверная карта — потеря времени, неверный указатель — не тот файл

Полноту контекст-пакета проверяют до правок

1. Попросить перечислить файлы к изменению
2. Попросить команды, которыми будет проверять
3. Список расходится — контекст неполный или постановка размыта
4. Пять минут сверки экономят час переделки

AGENTS.md — инструкция агенту, а не документация

1. Документация объясняет, как устроен проект и почему
2. AGENTS.md говорит, что делать и чего не делать
3. Команды проверки, политика веток, границы согласования
4. Короткая карта репозитория с указателями на подробности

Команды из AGENTS.md копируются в терминал дословно

1. Команды дословно, без пересказа
2. Порядок: ветка до правки, проверки до merge, merge до push
3. Запреты: миграции, `.env`, `docker-compose.yml`, workflow
4. Карта: backend, frontend, тесты, спеки, дизайн-система
5. Без пересказа архитектуры на десять экранов

Пробел в AGENTS.md фиксируют, но правку согласуют

1. Дважды объяснили одну команду — пробел в инструкции
2. Агент полез в прод-каталог — граница не записана
3. На этой неделе: формулировка и обоснование в `results.md`
4. Правку AGENTS.md принимает автор проекта

Сессия не помнит предыдущую — помнят файлы

1 **История сессии живёт только до её конца**

2 Постоянное: AGENTS.md, спека, план, `results.md`

3 `results.md`: что сделано, каким способом, что вышло, что осталось

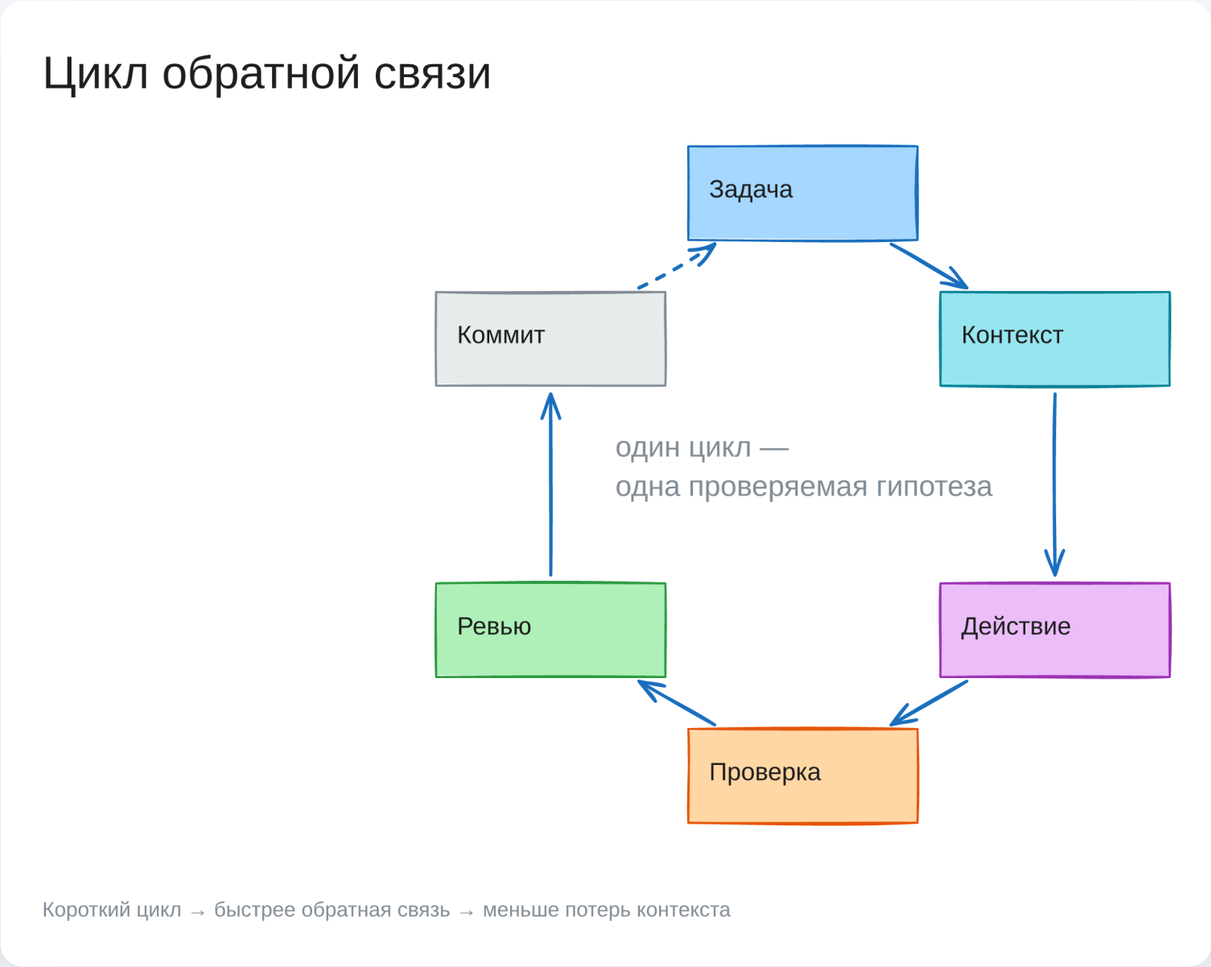
Компакция сохраняет смысл, но теряет детали

1. Harness сворачивает старые сообщения в summary
2. Теряются имена полей, номера строк, критерии, ссылки
3. Решения записывать сразу, а не надеяться на память
4. На неделе 4 — `.memory-bank/`: index, product, architecture, decisions

Feedback loop замыкается на командах, а не на заверениях

1. Цикл: сделал → проверил → прочитал → исправил
2. Без цикла агент не знает, компилируется ли код
3. Человек проверяет не «написал ли», а «какой командой подтверждено»
4. Команда не запускалась — результата нет, есть только текст

Цикл обратной связи



Цикл обратной связи

Проверки идут от дешёвых к дорогим

commands.sh

```
dotnet build CollegeLMS.slnx
dotnet csharpier check .
dotnet test CollegeLMS.slnx --filter FullyQualifiedName~{Name}
(cd CollegeLMS.Next && npm run build)
(cd CollegeLMS.Next && npm run check:design-system)
(cd CollegeLMS.Next && npx playwright test {spec})
```

- build → формат → таргетные тесты → frontend → гейты → E2E

Playwright допустим только на изолированном стенде

1. Worktree изолирует файлы, а не сервер и базу
2. `localhost:3000` и `reuseExistingServer` ведут на общий стенд
3. Нужны свой `PLAYWRIGHT_BASE_URL` и одноразовая база
4. Иначе в журнале: «не запускался: нет изолированного стенда»

Код возврата — не истина

1. Ноль — успех, ненулевой — ошибка, но этого мало
2. Первая ошибка — причина, остальные — следствия
3. Полный лог в файл, а не пересказ хвоста
4. Ложное зелёное: код 0, а проверено не то
5. Фильтр без совпадений, кэш, пропуск, не тот спек, скрытая ошибка
6. В журнал — что именно проверялось

Упавший тест: смотреть на первую ошибку

Терминал — dotnet test

```
$ dotnet test --nologo

  Determining projects to restore...
  All projects are up-to-date for restore.
  StudentFilterTests -> /tmp/student-filter-demo/bin/Debug/net10.0/StudentFilterTests.dll
Test run for /tmp/student-filter-demo/bin/Debug/net10.0/StudentFilterTests.dll (.NETCoreApp,Version=v10.0)
A total of 1 test files matched the specified pattern.
[xUnit.net 00:00:00.44]     CollegeLms.Tests.StudentFilterTests.Filters_by_group [FAIL]
  Failed CollegeLms.Tests.StudentFilterTests.Filters_by_group [14 ms]
  Error Message:
   Assert.Equal() Failure: Values differ
Expected: 3
Actual:   4
  Stack Trace:
     at CollegeLms.Tests.StudentFilterTests.Filters_by_group() in /tmp/student-filter-demo/StudentFilterTes...
   at System.Reflection.MethodBaseInvoker.InterpretedInvoke_Method(Object obj, IntPtr* args)
   at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj, BindingFlags invokeAttr)

Failed!  - Failed:     1, Passed:     6, Skipped:     0, Total:     7, Duration: 79 ms - StudentFilterTests...



Фактический прогон, не макет · 7 тестов: 6 прошли, 1 упал · код возврата 1
```

Фактический вывод dotnet test

Пять ошибок, которые обесценивают проверку

1. Проверять не то: frontend build при правке backend
2. Игнорировать формат: csharpier падает, а правят логику
3. Чинить тест вместо кода — потеря проверки
4. Гонять E2E на каждую правку текста
5. Работать в прод-каталоге: там контейнеры и пользователи

Зелёный build: код 0 и ни одной ошибки

dotnet-build.log

```
$ dotnet build CollegeLMS.slnx
...
Build succeeded.
    0 Warning(s)
    0 Error(s)
```

- Ожидаемый вывод; сборка в этой итерации не запускалась

Зелёный прогон — это выбранные и выполненные тесты

dotnet-test.log

```
$ dotnet test CollegeLMS.slnx --filter FullyQualifiedName~StudentFilterTests
Passed!  - Failed:     0, Passed:     7, Skipped:     0, Total:     7
```

- В журнал: cwd, фактическая команда, число тестов, код возврата, первая ошибка, путь к логу

Резюме: что запомнить

1. Контекст — минимум релевантного, а не весь репозиторий
2. Сначала карта и указатели, потом правки
3. AGENTS.md — команды, порядок, запреты, карта
4. Память — это файлы, а не история чата
5. Цикл замыкается на команде, а не на слове «готово»
6. Зелёный результат подтверждается логом и числом тестов

Вопросы для самопроверки

1. Почему «прочитай весь репозиторий» ухудшает результат?
2. Чем AGENTS.md отличается от документации проекта?
3. Что теряется при компакции и как это компенсировать?
4. Почему проверки идут от дешёвых к дорогим?
5. Какой прогон считается зелёным, а какой — ложным зелёным?
6. Как отличить ошибку кода от ошибки проверки?